Manual Implementation of a Multilayer Perceptron: Forward Pass, Backpropagation, and Optimizers

Shahyan Abbas[1]

National University of Computer and Emerging Sciences (NUCES),
Islamabad, Pakistan
i250143@isb.nu.edu.pk

Abstract. This report documents the complete manual implementation of a Multilayer Perceptron (MLP) with a 2–2–2–1 architecture, trained on a three-sample student-performance binary classification dataset. Working entirely by hand across seven structured tasks, the network was built from first principles: forward propagation, mean squared error loss, backpropagation via the chain rule, stochastic and mini-batch gradient descent, and two momentum-based optimizers—SGD with Momentum and Nesterov Accelerated Gradient (NAG). All intermediate values were computed to four decimal places at each step. To verify correctness across multi-iteration tasks including 5-step SGD (Task 4C), 3-epoch mini-batch gradient descent (Task 5B), and a 10-iteration optimizer comparison (Task 6C), a 561-line C++ program was written independently, replicating every computation including the project's rounding convention. A typographical error in the worksheet's backpropagation delta equations was identified during construction of the required computation graph, reported to the teaching assistant, and confirmed; the corrected equations were used throughout. The project concluded with a NumPy implementation scaled to MNIST (784 inputs, 128–64–10 neurons), trained using mini-batch gradient descent over 20 epochs, achieving a test accuracy of 95.54% on 10,000 held-out images.

Keywords: Multilayer perceptron · Backpropagation · Gradient descent · Mini-batch training · Momentum · Nesterov accelerated gradient · MNIST

1 Introduction

Artificial neural networks learn by iteratively adjusting their parameters to minimise a loss function. While modern deep learning frameworks automate this process, understanding the underlying mathematics is essential for diagnosing training failures, selecting appropriate optimizers, and extending existing architectures.

This project implements a Multilayer Perceptron (MLP) from first principles across seven tasks. The network has a 2–2–2–1 topology: two input features, two

hidden layers of two neurons each, and a single sigmoid output neuron. All computations in Tasks 1 through 6 were performed manually to four decimal places, replicating exactly what a neural network training loop executes at the scalar level.

To validate results across computationally intensive tasks—5-step SGD (Task 4C), 3-epoch mini-batch gradient descent (Task 5B), and the 10-iteration three-way optimizer comparison (Task 6C)—a C++ program was written independently. It encodes the full training loop including the 4-decimal rounding convention at every step, enabling direct comparison against hand-computed values. This was done for personal verification and understanding; the project itself required only manual worksheet completion.

Task 7 scales the same five functions to the MNIST handwritten digit dataset using NumPy, demonstrating that the identical mathematical structure generalises from 12 parameters to over 100,000 without conceptual change.

2 Background

2.1 Multilayer Perceptron Architecture

An MLP is a feedforward neural network in which each neuron computes a weighted sum of its inputs, adds a bias, and applies a non-linear activation function. For layer l and neuron j:

$$z_j^{(l)} = \sum_i w_{ji}^{(l)}\, a_i^{(l-1)} + b^{(l)}, \qquad a_j^{(l)} = \sigma\Big(z_j^{(l)}\Big) \tag{1}$$

In this project each hidden layer uses a single shared bias: $b^{(1)}$ is added to both h_1 and h_2, and $b^{(2)}$ is added to both h_3 and h_4. The output layer has no bias.

2.2 Sigmoid Activation

The sigmoid function maps any real input to $(0, 1)$:

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)\big(1-\sigma(z)\big) = a(1-a) \tag{2}$$

Its bounded output serves as a probability proxy for binary classification, and its closed-form derivative simplifies backpropagation.

2.3 Loss Functions

Four loss functions are relevant to this project.

Mean Absolute Error (MAE). $L_{\text{MAE}} = \frac{1}{m}\sum_{i=1}^{m} |y^{(i)} - \hat{y}^{(i)}|$. Robust to outliers but non-differentiable at zero.

Mean Squared Error (MSE).

$$L_{\text{MSE}} = \frac{1}{m}\sum_{i=1}^{m}\left(y^{(i)} - \hat{y}^{(i)}\right)^2 \tag{3}$$

Smooth and differentiable everywhere. Used in this project because $\partial L/\partial \hat{y} = -2(y - \hat{y})$ is straightforward to derive and apply by hand.

Cross-Entropy (CE). $L_{\text{CE}} = -\frac{1}{m}\sum_i \sum_k y_k^{(i)} \log \hat{y}_k^{(i)}$. Used with softmax outputs for multi-class classification.

Binary Cross-Entropy (BCE). $L_{\text{BCE}} = -\frac{1}{m}\sum_i \big[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})\big]$. The theoretically appropriate companion to sigmoid outputs for binary classification, penalising confident wrong predictions logarithmically rather than quadratically.

2.4 Backpropagation and the Chain Rule

Backpropagation computes $\partial L/\partial w$ for every weight by applying the chain rule from output to input. The error signal at each neuron is the delta:

$$\delta_j^{(l)} = \frac{\partial L}{\partial z_j^{(l)}} \tag{4}$$

The output-layer delta for MSE loss is:

$$\delta_1^{(3)} = -2(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \tag{5}$$

Hidden layer 2 deltas:

$$\delta_1^{(2)} = \left(w_9 \cdot \delta_1^{(3)}\right) \cdot a_1^{(2)}\left(1 - a_1^{(2)}\right) \tag{6}$$

$$\delta_2^{(2)} = \left(w_{10} \cdot \delta_1^{(3)}\right) \cdot a_2^{(2)}\left(1 - a_2^{(2)}\right) \tag{7}$$

For hidden layer 1, the correct equations—derived from the Task 1A weight labelling and network diagram—are:

$$\delta_1^{(1)} = \left(w_5 \cdot \delta_1^{(2)} + w_6 \cdot \delta_2^{(2)}\right) \cdot a_1^{(1)}\left(1 - a_1^{(1)}\right) \tag{8}$$

$$\delta_2^{(1)} = \left(w_7 \cdot \delta_1^{(2)} + w_8 \cdot \delta_2^{(2)}\right) \cdot a_2^{(1)}\left(1 - a_2^{(1)}\right) \tag{9}$$

The weight gradient is $\partial L/\partial w_{ji}^{(l)} = \delta_j^{(l)} \cdot a_i^{(l-1)}$, and the shared bias gradient is $\partial L/\partial b^{(l)} = \sum_j \delta_j^{(l)}$.

Note on a worksheet error. The delta equations printed in the Task 3 worksheet contained a typographical error: w_7 was used in the expression for

$\delta_1^{(1)}$ and w_6 in the expression for $\delta_2^{(1)}$. Per the Task 1A equations, w_5 connects $h_1 \to h_3$, w_6 connects $h_1 \to h_4$, w_7 connects $h_2 \to h_3$, and w_8 connects $h_2 \to h_4$. The backward pass through h_1 must therefore accumulate error via w_5 and w_6—not w_5 and w_7. This error was identified while constructing the required backpropagation computation graph on page 13 of the worksheet, reported to the teaching assistant, and confirmed. All calculations in this report use the corrected Equations (8)–(9).

2.5 Gradient Descent Variants

Batch Gradient Descent (BGD). Averages gradients over all m samples before each update. Stable but slow on large datasets.

Stochastic Gradient Descent (SGD). Updates weights after each individual sample. Fast but high-variance gradient estimates yield a noisy loss curve. Used in Tasks 3 and 4.

Mini-Batch Gradient Descent (MBGD). Averages gradients over a batch of size B:

$$\frac{\partial L}{\partial w} = \frac{1}{B} \sum_{i \in \text{batch}} \frac{\partial L^{(i)}}{\partial w}, \qquad w \leftarrow w - \eta \cdot \frac{\partial L}{\partial w} \tag{10}$$

The industry standard, balancing update frequency against gradient variance.

2.6 Optimizers

SGD with Momentum. Maintains a velocity v_t as an exponentially weighted average of past gradients:

$$v_t = \beta\, v_{t-1} + (1-\beta)\, g_t, \qquad w \leftarrow w - \eta\, v_t \tag{11}$$

where $\beta = 0.9$, $g_t = \partial L/\partial w$, and $v_0 = 0$.

Nesterov Accelerated Gradient (NAG). Evaluates the gradient at a lookahead position:

$$\tilde{w} = w - \eta\beta\, v_{t-1} \tag{12}$$

$$g_t = \frac{\partial L(\tilde{w})}{\partial \tilde{w}} \tag{13}$$

$$v_t = \beta\, v_{t-1} + (1-\beta)\, g_t, \qquad w \leftarrow w - \eta\, v_t \tag{14}$$

When $v_0 = 0$, the lookahead equals the current position, so NAG is numerically identical to Momentum on the first step.

3 Implementation

3.1 Network Architecture

Tables 1 and 2 summarise the dataset and architecture used in Tasks 1–6.

Table 1. Student-performance dataset (Tasks 1–6).

Sample	x_1 (Study Hours)	x_2 (Attendance)	y (Pass?)
$s^{(1)}$	0.2	0.8	1
$s^{(2)}$	0.9	0.4	1
$s^{(3)}$	0.1	0.2	0

Table 2. MLP architecture for Tasks 1–6.

Layer	Type	Neurons	Activation
Layer 0	Input	2	–
Layer 1	Hidden layer 1	2	Sigmoid
Layer 2	Hidden layer 2	2	Sigmoid
Layer 3	Output	1	Sigmoid

3.2 Forward Pass

Hidden layer 1:

$$z_1^{(1)} = w_1 x_1 + w_3 x_2 + b^{(1)}, \qquad a_1^{(1)} = \sigma(z_1^{(1)}) \tag{15}$$

$$z_2^{(1)} = w_2 x_1 + w_4 x_2 + b^{(1)}, \qquad a_2^{(1)} = \sigma(z_2^{(1)}) \tag{16}$$

Hidden layer 2:

$$z_1^{(2)} = w_5 a_1^{(1)} + w_7 a_2^{(1)} + b^{(2)}, \qquad a_1^{(2)} = \sigma(z_1^{(2)}) \tag{17}$$

$$z_2^{(2)} = w_6 a_1^{(1)} + w_8 a_2^{(1)} + b^{(2)}, \qquad a_2^{(2)} = \sigma(z_2^{(2)}) \tag{18}$$

Output (no bias):

$$z_1^{(3)} = w_9 a_1^{(2)} + w_{10} a_2^{(2)}, \qquad \hat{y} = \sigma(z_1^{(3)}) \tag{19}$$

3.3 Backpropagation

Gradients were computed using the corrected delta equations from Section 2.4. The 12 gradients from Task 3C were used directly in all subsequent weight updates.

3.4 Training Procedure

Tasks 3–4. Gradients computed using only $s^{(1)}$, five iterations, $\eta = 0.1$.

Task 5. Three epochs, $B = 2$, fixed shuffle order from the worksheet. Batch MSE computed over batch samples only. Weights reset to W_0 before epoch 1.

Task 6. Three optimizers run from W_0 for 10 iterations, each iteration being one full mini-batch epoch ($B = 2$).

Task 7. MNIST, 784 inputs, 128–64–10 architecture, $B = 32$, $\eta = 0.1$, 20 epochs, `seed = 42`. The NumPy backpropagation function:

Listing 1.1. Backpropagation in NumPy (Task 7).

```
def backpropagation(X, Y_true, Z1, A1, Z2, A2, Z3, A3,
                    W1, W2, W3):
    m = X.shape[0]
    delta3 = -2 * (Y_true - A3) * sigmoid_derivative(A3)
    dW3 = (A2.T @ delta3) / m
    db3 = np.sum(delta3, axis=0, keepdims=True) / m
    delta2 = (delta3 @ W3.T) * sigmoid_derivative(A2)
    dW2 = (A1.T @ delta2) / m
    db2 = np.sum(delta2, axis=0, keepdims=True) / m
    delta1 = (delta2 @ W2.T) * sigmoid_derivative(A1)
    dW1 = (X.T @ delta1) / m
    db1 = np.sum(delta1, axis=0, keepdims=True) / m
    return dW1, db1, dW2, db2, dW3, db3
```

4 Experiments

4.1 Hyperparameters

Table 3. Hyperparameters across all tasks.

Parameter	Tasks 1–6	Task 7 (MNIST)
Learning rate η	0.1	0.1
Batch size B	2 (Tasks 5–6)	32
Momentum β	0.9 (Task 6)	–
Epochs	3 (Task 5), 10 iters (Task 6)	20
Weight init	Assigned values	Uniform $[-0.5, 0.5]$
Bias init	Assigned values	0
Random seed	–	42

4.2 Task 4C: SGD over 5 Iterations

The MSE loss decreased consistently and smoothly across all five iterations. The smooth curve is explained by the experimental setup: the same sample $s^{(1)}$ was used for every gradient update, so the gradient direction was constant across iterations. This eliminates the variance that makes true SGD noisy, making the Task 4C run more analogous to gradient descent on a single-sample dataset than to genuine stochastic gradient descent.

4.3 Task 5B/5C: Mini-Batch GD over 3 Epochs

Six batch updates were performed across three epochs. Comparing the Task 5C mini-batch curve against the Task 4C SGD curve, the SGD curve appeared smoother. This is a direct consequence of the fixed-sample SGD setup: because $s^{(1)}$ was always used, the gradient direction never changed and the loss decreased monotonically. The mini-batch updates, drawing on different sample combinations each batch, produced gradient directions that varied across update steps—which is expected and desirable behaviour, as it provides a less biased estimate of the true gradient over the full dataset.

4.4 Task 6C: Optimizer Comparison over 10 Iterations

Table 4. MSE loss over 10 iterations for three optimizers ($\eta = 0.1$, $\beta = 0.9$, $B = 2$).

Iteration	Plain GD	Momentum	NAG
1	0.3080	0.3086	0.3086
3	0.3030	0.3074	0.3074
5	0.2999	0.3052	0.3052
7	0.2970	0.3023	0.3023
10	0.2922	0.2978	0.2978

Plain GD reached the lowest final loss (0.2922) after 10 iterations, with both Momentum and NAG finishing at 0.2978. This outcome is consistent with the pattern observed in Tasks 4C and 5C and attributable to the same underlying factors. The three-sample dataset produces an extremely smooth loss surface with no flat regions or saddle points—precisely the terrain where momentum accumulation provides no benefit. With the scaled variant ($v_t = \beta v_{t-1} + (1 - \beta)g_t$), the effective step at early iterations is $(1 - \beta)g_t = 0.1g_t$, which is $10\times$ smaller than the plain GD step g_t. Velocity builds up gradually, and over only 10 iterations the compounding advantage of momentum has not had enough time to overcome this initial lag. On a larger dataset, a longer training run, or a loss surface with genuine geometric complexity, both Momentum and NAG would be expected to outperform plain GD.

Momentum and NAG produced identical losses for iterations 1–7, with only a marginal four-decimal-place difference at iteration 8 (NAG: 0.3010, Momentum: 0.3009), converging again by iteration 9. This is expected: NAG's look-ahead is proportional to $\eta\beta v_{t-1}$, which is negligibly small when velocity has not yet accumulated.

4.5 Task 7: MNIST Results

Loss curve. Training MSE decreased from 0.0296 (epoch 1) to 0.0069 (epoch 20), with consistent improvement at every epoch (Table 5). The curve (Figure 1)

shows rapid initial improvement that gradually decelerates, characteristic of gradient descent on a well-conditioned problem.

Table 5. Training MSE loss per epoch on MNIST (20 epochs).

Epoch	Loss	Epoch	Loss
1	0.0296	11	0.0095
2	0.0208	12	0.0091
3	0.0172	13	0.0088
4	0.0153	14	0.0084
5	0.0138	15	0.0081
6	0.0127	16	0.0078
7	0.0119	17	0.0076
8	0.0111	18	0.0073
9	0.0105	19	0.0071
10	0.0100	20	0.0069

Test accuracy. The network achieved **95.54%** accuracy on 10,000 held-out MNIST test images. This is strong performance for a sigmoid-activated MLP trained with plain MSE loss and no regularisation, confirming that the NumPy implementation of the manual-task equations is correct and effective at scale.

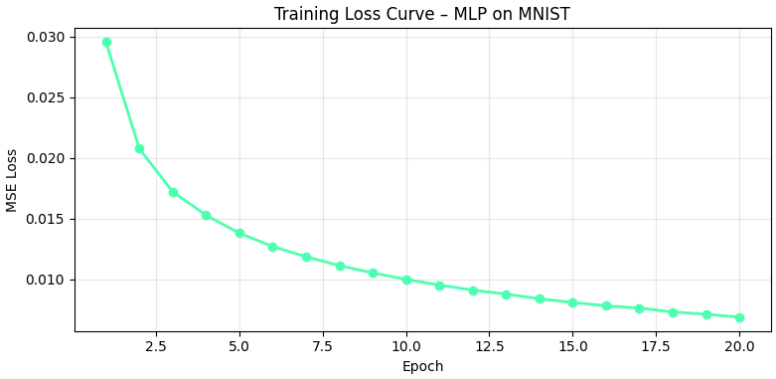

Fig. 1. Training MSE loss vs. epoch for the MNIST MLP (20 epochs, η=0.1, B=32).

5 Discussion

Consistency between manual and NumPy implementations. The same five operations—sigmoid, forward pass, MSE loss, backpropagation, weight update—were com-

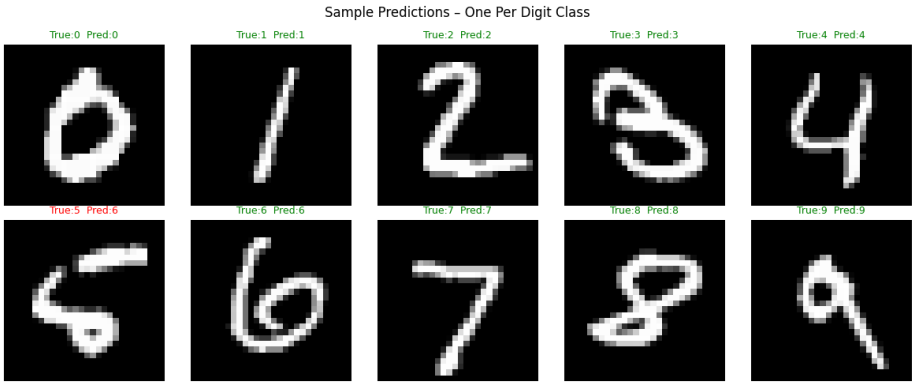

Fig. 2. Sample predictions on the MNIST test set, one image per digit class. Green titles indicate correct predictions.

puted by hand at the scalar level and then translated directly into NumPy. The structural identity between the two confirms that the manual calculations are exact representations of what the code executes, not approximations.

Why plain GD outperformed momentum-based methods. As discussed in Section 4.3, the result stems from the smooth, low-dimensional loss surface, the scaled momentum formulation reducing early effective step size, and insufficient iterations for velocity to compound meaningfully. This result should not be interpreted as evidence against momentum in general.

SGD with fixed sample vs. true SGD. The smooth, monotonically decreasing Task 4C loss curve is an artefact of the fixed-sample experimental design. True SGD, which selects samples randomly, produces noisy curves because gradient direction changes with each sample. The Task 4C setup is better described as gradient descent on a one-sample dataset.

Vanishing gradients. The sigmoid derivative $\sigma'(z) \leq 0.25$ causes gradient magnitudes to shrink at each layer. Over two hidden layers, the gradient at layer 1 can be up to $16\times$ smaller than at the output. This motivates ReLU activations and batch normalisation in deeper networks.

Worksheet typographical error. The delta equations on page 15 of the worksheet used w_7 in $\delta_1^{(1)}$ and w_6 in $\delta_2^{(1)}$. Applying the general formula $\delta_j^{(l)} = \sum_k w_{kj}^{(l+1)} \delta_k^{(l+1)} \cdot a_j^{(l)}(1 - a_j^{(l)})$ to h_1 requires summing over connections *originating from* h_1, which are w_5 (to h_3) and w_6 (to h_4). The printed equations were inconsistent with this, and the error was caught through the required computation graph construction.

6 Conclusion

This project demonstrated a complete manual-to-code implementation of an MLP, building concrete intuition for every step in a neural network training loop. Key findings:

- The delta method scales identically from scalar hand computation to vectorised NumPy matrix operations without any change in mathematical structure.
- On a three-sample dataset, plain gradient descent outperforms momentum-based methods due to the smooth loss surface and short training run; this result does not generalise.
- NAG and Momentum are numerically near-identical over 10 iterations because look-ahead requires meaningful accumulated velocity to produce different gradients.
- The MNIST implementation achieved 95.54% test accuracy after 20 epochs, confirming correctness and effective scaling of the manually derived equations.
- Careful backpropagation graph construction is necessary to catch inconsistencies between printed equations and the underlying architecture.

References

1. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press, Cambridge, MA (2016). https://www.deeplearningbook.org
2. Boyd, S., Vandenberghe, L.: Convex Optimization. Cambridge University Press, New York, NY (2004). https://web.stanford.edu/~boyd/cvxbook/